

Instituto Politécnico Nacional

Centro de Investigación en Computación (CIC)

Departamento de diplomados y extensión profesional

Curso de Inteligencia Artificial con Python

Profesor Alan Badillo Salas

Laboratorio

Agentes IA con ChatGPT

Semana 6. Febrero 28, 2026.

1. Introducción

En este laboratorio se construyen distintos tipos de agentes basados en modelos de lenguaje de OpenAI utilizando Python.

Se exploran cinco capacidades fundamentales:

- Generación de texto con prompts libres.
- Uso de *function calling* para estructurar datos.
- Construcción de búsquedas estructuradas.
- Generación de imágenes.
- Extracción de información jurídica desde texto y archivos.

El objetivo es comprender cómo transformar un modelo generativo en un agente capaz de producir salidas estructuradas y ejecutables.

2. Acceso seguro a la API

Para consumir los modelos es necesario utilizar una clave privada. En Google Colab se puede almacenar en `userdata`.

Obtención de la clave

```
from google.colab import userdata

apikey = userdata.get("chatgpt_key")

print(apikey[:20] + "...")
```

3. Agente generador de texto

Se construye un agente simple que genera una rutina de entrenamiento a partir de una descripción libre del usuario.

```
from openai import OpenAI

client = OpenAI(api_key=apikey)

prompt = input("Describe tu objetivo de entrenamiento: ")

response = client.responses.create(
    model="gpt-4.1-mini",
    input=f"""
```

```

    """Prepara una rutina de ejercicios para
    una rutina de entrenamiento:

    """{prompt}
    """
)

print(response.output_text)

```

4. Agente con llamadas a función

El modelo puede estructurar información y devolver argumentos listos para ejecutar funciones del sistema.

```

tools = [
    {
        "type": "function",
        "name": "crear_pedido",
        "description": "Crea un pedido de comida para un cliente",
        "parameters": {
            "type": "object",
            "properties": {
                "cliente": {"type": "string"},
                "items": {
                    "type": "array",
                    "items": {
                        "type": "object",
                        "properties": {
                            "producto": {"type": "string"},
                            "cantidad": {"type": "integer"},
                            "notas": {"type": "string"}
                        },
                        "required": ["producto", "cantidad"]
                    }
                }
            },
            "required": ["items"]
        }
    }
]

prompt = input("Explica tu pedido: ")

response = client.responses.create(
    model="gpt-4.1-mini",

```

```

    input=prompt,
    tools=tools
)

```

5. Construcción de búsquedas estructuradas

Se construye un agente que traduce lenguaje natural en criterios estructurados de consulta.

```

tools = [
    {
        "type": "function",
        "name": "buscar_producto",
        "description": "Busca productos en el catalogo",
        "parameters": {
            "type": "object",
            "properties": {
                "categoria": {"type": "string"},
                "marca": {"type": "string"},
                "color": {"type": "string"},
                "precio_min": {"type": "number"},
                "precio_max": {"type": "number"},
                "atributos": {
                    "type": "array",
                    "items": {"type": "string"}
                },
                "texto_libre": {"type": "string"}
            },
        }
    }
]

prompt = input("Explica que deseas buscar: ")

response = client.responses.create(
    model="gpt-4.1-mini",
    input=prompt,
    tools=tools
)

```

6. Generación de imágenes

Se utiliza el modelo generador de imágenes para producir contenido visual a partir de texto.

```

from openai import OpenAI
import base64

client = OpenAI(api_key=apikey)

prompt = input("Describe la imagen a generar: ")

result = client.images.generate(
    model="gpt-image-1",
    prompt=prompt,
    size="1024x1024"
)

image_base64 = result.data[0].b64_json

```

7. Extracción estructurada desde documentos jurídicos

Se diseña un agente capaz de leer texto jurídico y devolver información estructurada.

```

tools = [
    {
        "type": "function",
        "name": "extraer_sentencia",
        "description": "Extrae información estructurada",
        "parameters": {
            "type": "object",
            "properties": {
                "ciudad": {"type": "string"},
                "fecha_sentencia": {"type": "string"},
                "numero_expediente": {"type": "string"},
                "tipo_juicio": {"type": "string"},
                "actor": {"type": "string"},
                "demandado": {"type": "string"},
                "accion_principal": {"type": "string"},
                "monto_contrato": {"type": "number"},
                "fecha_contrato": {"type": "string"},
                "resolucion": {"type": "string"}
            },
            "required": ["fecha_sentencia", "numero_expediente"]
        }
    }
]

```

8. Conclusiones

En este laboratorio se observaron distintos niveles de sofisticación en agentes IA:

- Generación libre (modo asistente).
- Estructuración de datos (function calling).
- Traducción semántica a filtros formales.
- Generación multimodal.
- Extracción de conocimiento desde documentos.

El paso clave en la construcción de agentes no es únicamente generar texto, sino transformar el lenguaje natural en acciones ejecutables.